Sub-Millijoule Intrusion Detection on a Commodity MCU Neural Processing Unit

Anonymous
Double-blind review

***Abstract*—We deploy an intrusion detection system (IDS) classifier on the STM32N6570-DK, a commodity ARM Cortex-M55 MCU with the Neural-ART NPU rated at 600 GOPS INT8. Exploiting the approximate equivalence between single-timestep (T=1) spiking neural network (SNN) inference and INT8 quantized ANN inference, we compile a lightweight MLP to the NPU without neuromorphic hardware. On NSL-KDD (5-class) and UNSW-NB15 (10-class) with 10 random seeds, the model achieves 78.86 ± 1.32% and 64.75 ± 0.61% overall accuracy, respectively. INT8 NPU inference completes in 0.29–0.46 ms (2.7–4.2× faster than CPU-only), with an estimated energy of 44–69 µJ/inference and a Flash footprint of 120–138 KB. Compared with recent MCU-class IDS deployments on the STM32F7 (31 ms, 7.86 mJ) and Raspberry Pi 3 (27 ms), our NPU path achieves 60–107× lower latency and over 100× lower energy per inference. To our knowledge, this is the first publicly documented IDS deployment on an ARM Cortex-M NPU, and also the first non-vision workload reported on the Neural-ART accelerator.**

***Index Terms*—intrusion detection, spiking neural network, neural processing unit, INT8 quantization, edge AI, STM32N6**

I. Introduction

Deep-learning-based intrusion detection at the IoT edge must operate under tight power and latency budgets. Recent MCU-class deployments achieve promising accuracy but at high energy cost: Chehade et al. reported 31 ms and 7.86 mJ per inference on an STM32F746G (Cortex-M7, no NPU) [1], and Diab et al. required 27 ms on a Raspberry Pi 3B+ [2]. At these latencies, CPU-only inference blocks all concurrent RTOS tasks for tens of milliseconds, making real-time packet processing difficult.

Commodity MCUs are beginning to integrate neural processing units (NPUs) that execute INT8 matrix operations at hundreds of GOPS while freeing the CPU for other tasks. A line of theoretical work [3]–[5] has shown that single-timestep (T=1) SNN inference is approximately equivalent to INT8 quantized ANN inference: when the membrane potential is initialized to zero, the threshold-and-fire mechanism reduces to a ReLU-like clamp. Any NPU executing INT8 `Gemm`+`Relu` can therefore run an SNN-equivalent model.

We validate this on the STM32N6570-DK (ARM Cortex-M55 @ 800 MHz, Neural-ART NPU, 600 GOPS INT8, $89 dev kit). We focus on classifier inference latency, energy, and memory footprint on pre-extracted flow-level features; the full IDS pipeline (packet capture, flow aggregation) is left to future work.

Our contributions:

TABLE I
Hardware-deployed IDS comparison. Energy is per inference.

Work	Platform	Task	Acc.	Lat.	Energy	NPU
Ngo [6]	MAX78000	bin.	98.6%	—	18 mW*	CNN eng.
Zahm [7]	Akida	multi	98.4%	—	∼1 W*	Neurom.
Chehade [1]	STM32F7	multi	96.6%	31 ms	7.86 mJ	None
Diab [2]	RPi 3B+	multi	95.3%	27 ms	∼6.75 mJ	None
This work	**STM32N6**	**5/10**	**78.9/64.8%**†	**0.29 ms**	**44 µJ**	**NPU**

*Power, not energy; latency not reported.
†NSL-KDD 5-class / UNSW-NB15 10-class.

- To our knowledge, the first publicly documented IDS classifier deployment on an ARM Cortex-M NPU, achieving 0.29–0.46 ms inference at an estimated 44–69 µJ.
- Empirical validation of T=1 SNN–ANN equivalence on commercial NPU silicon, with 99% prediction agreement between FP32 and INT8 models.
- An operator-level analysis showing that the `Floor` operator required by QCFS [3] falls back to CPU, adding 17.6% latency.

II. Related Work

Table I compares IDS deployments on physical hardware. Ngo et al. [6] deployed a 1,360-parameter MLP on the MAX78000, an AI-specialized MCU with a fixed CNN accelerator, achieving 98.57% on UNSW-NB15 (binary) at 18 mW. Zahm et al. [7] used the BrainChip Akida AKD1000 neuromorphic processor (98.4%, ∼1 W). Both use purpose-built AI/neuromorphic hardware. Chehade et al. [1] deployed a 1D-CNN on the STM32F746G (Cortex-M7, no NPU), achieving 96.59% accuracy on ISCX VPN-nonVPN but requiring 31 ms and 7.86 mJ per inference. Diab et al. [2] deployed LightGBM/CNN on Raspberry Pi 3B+ (27–300 ms, 250–275 mW). Our work targets a general-purpose MCU with an attached programmable NPU, achieving two orders of magnitude lower energy than CPU-only MCU deployments.

Several GPU-based SNN-IDS systems have also been proposed: convolutional SNNs [8], event-driven SNNs [9], and transformer-spiking hybrids [10] report 94–99% accuracy on NSL-KDD/UNSW-NB15 but use binary classification and have no path to edge deployment. Our multi-class formulation (5/10-class) and hardware deployment target a different design point.

III. System Design

A. T=1 SNN–ANN Equivalence

A Leaky Integrate-and-Fire (LIF) neuron updates its membrane potential $V[t] = \beta \cdot V[t-1] + \mathbf{W}\mathbf{x}[t]$ and fires when $V[t] > \theta$. At T=1 with $V[0]$=0, the leak term vanishes and firing reduces to $\Theta(\mathbf{W}\mathbf{x} - \theta)$, which is equivalent to a quantized ReLU. Bu et al. [3] formalized this via QCFS; Jiang et al. [4] and Bu et al. [5] further showed that INT8 post-training quantization produces a discretization grid that closely approximates the threshold-and-fire operation. In practice, any NPU that runs INT8 `Gemm+Relu` is executing SNN-equivalent computation.

B. Neural-ART NPU

The STM32N6570-DK pairs an ARM Cortex-M55 at 800 MHz with the Neural-ART NPU rated at 600 GOPS for INT8 and 3 TOPS/W efficiency [11]. The NPU supports `Conv`, `Gemm`, `Relu`, `Add`, `Clip`, and `MaxPool` in INT8; unsupported operators fall back to the Cortex-M55 CPU with automatic cache-maintenance overhead at each NPU↔CPU handoff. The dev kit costs $89 and includes 4.2 MB internal SRAM and 128 MB external NOR Flash.

C. Model Architecture

We use a four-layer MLP with BatchNorm and ReLU: `Lin`(d→256) → `BN` → `ReLU` → `Lin`(256→256) → `BN` → `ReLU` → `Lin`(256→128) → `BN` → `ReLU` → `Lin`(128→C), where d=41 (NSL-KDD) or 34 (UNSW-NB15) and C=5 or 10. BatchNorm is folded into the preceding Linear at ONNX export, yielding a graph of `Gemm+Relu` operators only (110–111K parameters). The architecture is deliberately shallow: deeper models could improve minority-class recall but risk introducing operators outside the NPU's supported set. All hidden dimensions are powers of two (256, 128), avoiding a known Neural-ART compiler issue with prime-number channel counts [11].

D. Datasets and Training

NSL-KDD [12]: 125,973 train / 22,544 test, 41 features, 5-class. **UNSW-NB15** [13]: 175,341 train / 82,332 test, 34 features, 10-class. Training: Adam (lr=10^{-3}), cosine annealing, 80 epochs, inverse-frequency class weighting, 10 seeds. Three categorical features are label-encoded; remaining features are z-score normalized using training-set statistics. FP32 models are quantized to INT8 via ONNX Runtime static PTQ (MinMax, per-tensor, 1,000 calibration samples). A 24-configuration ablation (Section IV-C) confirms that the choice of calibration method, granularity, and sample size changes accuracy by at most 0.82 pp.

E. NPU Compilation and Operator Mapping

Models are compiled for the STM32N6570-DK using ST Edge AI Developer Cloud v4.0.0 (`atonn` v1.1.3). Table II shows the operator-level mapping. All `Gemm+Relu` layers map to NPU hardware epochs. The `QuantizeLinear`/`DequantizeLinear` operators at the model boundary may require CPU epochs depending on the input dimension.

TABLE II
Operator mapping to NPU epochs. ReLU INT8 achieves near-100% NPU utilization; QCFS INT8 forces CPU fallback at every `Floor`.

ONNX Operator	NPU Support	Epoch Type	Notes
`Gemm` (INT8)	✓	HW	Fused weight + bias
`Relu`	✓	HW	Fused with preceding Gemm
`Clip`	✓	HW	Used by QCFS
`Floor`	✗	SW (float)	CPU fallback, +0.08 ms
`QuantizeLinear`	✓	HW/Hyb	At model boundary
`DequantizeLinear`	✓	HW/Hyb	At model boundary

TABLE III
Test accuracy (%, mean±std, 10 seeds). MLP is the only model eligible for NPU acceleration.

Dataset	Model	Overall	Macro F1	NPU?
NSL-KDD	ReLU MLP	**78.86±1.32**	**59.20±2.80**	✓
	QCFS MLP	77.82±1.11	57.29±2.77	Partial
	Rand. Forest	73.84±0.19	47.13±0.33	✗
UNSW-NB15	ReLU MLP	64.75±0.61	40.29±0.90	✓
	Rand. Forest	**69.46±0.10**	**48.63±0.38**	✗

For the ReLU INT8 path: NSL-KDD compiles to 5 HW + 1 Hyb + 2 SW epochs (0.46 ms); UNSW-NB15 compiles to 4 HW + 0 Hyb + 0 SW epochs (0.29 ms, 100% NPU). The QCFS INT8 path compiles to 13 HW + 1 Hyb + 14 SW epochs (0.54 ms) because each `Floor` triggers a CPU round-trip via `Dequant`→`Floor`→`Quant`.

IV. Experiments

A. Classification Results

Table III reports multi-seed results. On NSL-KDD, the ReLU MLP outperforms both QCFS (Wilcoxon p=0.037) and tree baselines. On UNSW-NB15 (10-class), Random Forest achieves higher overall accuracy, but cannot be compiled for the NPU: ST Edge AI Core rejects `TreeEnsembleClassifier` with "NOT IMPLEMENTED." The accuracy gap reflects the difficulty of 10-class classification with extreme imbalance (Worms: 44 samples), not a modeling limitation specific to NPU deployment.

Per-class analysis on NSL-KDD reveals that DoS (F1=87.3%) and Normal (82.6%) are well-classified, while R2L suffers from low recall (25.0%) — 64.4% of R2L samples are misclassified as Normal. U2R (200 test samples) scatters across Probe (50.9%) and Normal (24.0%). On UNSW-NB15, Generic achieves F1=97.9%, but minority classes collapse: Worms F1=9.0%, Analysis F1=2.6%. These patterns are characteristic of extreme class imbalance, not NPU-induced degradation — FP32 and INT8 models agree on 99% of predictions.

Fig. 1 visualizes these patterns. The class imbalance is severe: R2L has only 995 training samples (0.8% of the set) yet 2,754 test samples (12.2%); U2R has 52 training samples.

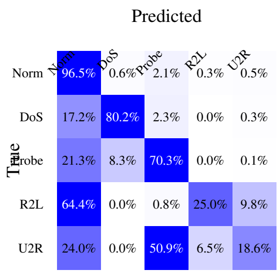

Fig. 1. NSL-KDD confusion matrix (%, 10-seed mean, ReLU INT8). R2L→Normal (64.4%) and U2R→Probe (50.9%) dominate errors.

TABLE IV
NPU BENCHMARK ON STM32N6570-DK. ENERGY ESTIMATED FROM AN5946 [14] (∼150 MW AT NOMINAL FREQUENCY); FP32 CPU-ONLY POWER DIFFERS AND IS NOT ESTIMATED (—).

Model	Dataset	Time (ms)	Energy (μJ)	HW	Hyb	SW	Flash (KB)	RAM (KB)
ReLU FP32	NSL-KDD	1.24	—	0	0	11	466.4	2.17
ReLU INT8	NSL-KDD	0.46	69	5	1	2	137.7	1.25
ReLU FP32	UNSW-NB15	1.23	—	0	0	11	461.9	2.14
ReLU INT8	UNSW-NB15	**0.29**	**44**	4	0	0	120.6	0.50
QCFS INT8	NSL-KDD	0.54	81	13	1	14	138.0	2.00

B. Energy, Latency, and Memory

Table IV presents benchmark results from ST Edge AI Developer Cloud, which cycle-accurately simulates the STM32N6570-DK. INT8 NPU execution yields **2.7**× speedup on NSL-KDD and **4.2**× on UNSW-NB15 over the same model on the Cortex-M55 CPU. At an estimated 150 mW (ST AN5946 [14]), this translates to 44–69 μJ per inference — **over 100**× less than Chehade et al.'s 7.86 mJ on the STM32F7 [1]. UNSW-NB15 achieves 100% NPU execution (4 HW, 0 SW epochs); Flash shrinks 3.4–3.8× from FP32 to INT8.

The energy figure is derived from the reference workload in AN5946, not direct measurement of our model. On-board validation with an STLINK-V3PWR probe is left to future work.

C. INT8 Quantization Robustness

A 24-configuration ablation (3 calibration methods × 2 granularities × 4 sample sizes) shows that INT8 accuracy deviates from FP32 by at most 0.82 pp across all configurations. Several INT8 configurations slightly *improve* over FP32, likely due to regularization effects of quantization noise.

Table V presents layer-wise activation comparison between FP32 and INT8 models on 1,000 NSL-KDD test samples. Hidden layers show moderate cosine similarity (0.65–0.68) due to per-neuron quantization noise, but the logit layer recovers to 0.978 and the overall prediction agreement is 99.0%. INT8 maps each activation to one of 256 discrete levels; the bounded per-neuron noise does not propagate into classification errors. From the $T{=}1$ SNN perspective, the quantized model (INT8) closely reproduces the firing decisions of the FP32 model.

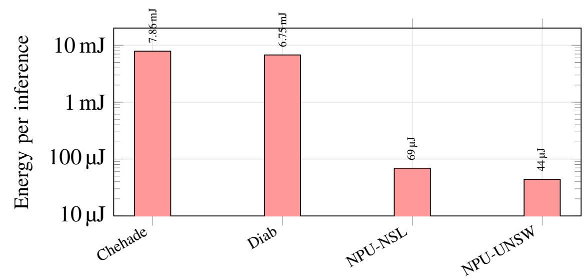

Fig. 2. Energy per inference (log scale). Our NPU path achieves over 100× reduction versus prior MCU-class deployments. NPU = ReLU INT8 on Neural-ART (estimated from AN5946). Chehade [1]: STM32F7; Diab [2]: RPi 3B+.

TABLE V
LAYER-WISE FP32 VS INT8 COMPARISON (1,000 NSL-KDD SAMPLES).

Layer	Cosine Sim	MAE	L_∞
Relu_0 (256)	0.667	0.435	43.7
Relu_1 (256)	0.655	0.438	62.6
Relu_2 (128)	0.683	0.400	65.2
Logits (5)	**0.978**	0.103	19.8

V. Discussion and Conclusion

Design-space positioning. This work targets a different point from GPU-based IDS: sub-millisecond, sub-millijoule inference on a $89 MCU. The absolute accuracy (78.9% NSL-KDD, 64.8% UNSW-NB15) is below GPU-simulation results, reflecting multi-class evaluation (5/10-class vs. binary) and NPU operator constraints (shallow MLP). Improving accuracy within these constraints is orthogonal to the deployment findings.

Why NPU over CPU? NPU inference frees the Cortex-M55 for RTOS tasks (packet capture, feature extraction), whereas CPU-only inference blocks the processor for 1.2 ms. Tree models achieve higher accuracy on UNSW-NB15 but have no execution path on the NPU (`TreeEnsembleClassifier` is not implemented in ST Edge AI Core).

QCFS barrier. The `Floor` operator is absent from the Neural-ART operator set, forcing CPU fallback with 17.6% latency overhead. ReLU INT8 remains the optimal activation for commodity MCU NPUs.

RTOS implications. At 0.29–0.46 ms, NPU inference fits within a typical 1 ms RTOS tick, and the Cortex-M55 remains free for concurrent tasks (Ethernet DMA, feature extraction) during NPU execution. The 1.2 ms CPU-only path would block the processor across tick boundaries. Validating this with μT-Kernel 3.0 is planned future work.

Limitations. We measure latency via ST Cloud benchmark; energy is estimated from the AN5946 application note, not direct measurement of our model. The evaluation covers classifier inference on pre-extracted features, not the complete IDS pipeline. NSL-KDD is a legacy dataset, and UNSW-NB15 has extreme class imbalance that limits absolute accuracy.

Conclusion. We demonstrated sub-millisecond, sub-millijoule IDS inference on a commodity MCU NPU, achieving 2.7–4.2$\times$ CPU speedup and over 100$\times$ energy reduction versus prior MCU-class deployments. Layer-wise analysis confirms 99% FP32/INT8 prediction agreement, validating $T{=}1$ SNN–ANN equivalence on commercial NPU silicon. Code and trained models will be released upon acceptance.

References

[1] M. Chehade *et al.*, "Energy-efficient deep learning for traffic classification on microcontrollers," in *Proc. IEEE ISCC*, 2025, arXiv:2506.10851.

[2] F. A. Diab *et al.*, "Intrusion detection on resource-constrained IoT devices with hardware-aware ML and DL," *arXiv preprint arXiv:2512.02272*, 2025.

[3] T. Bu, W. Fang, J. Ding, P. Dai, Z. Yu, and T. Huang, "Optimal ANN-SNN conversion for high-accuracy and ultra-low-latency spiking neural networks," in *Proc. ICLR*, 2022.

[4] H. Jiang, S. Peng *et al.*, "A unified optimization framework of ANN-SNN conversion: Towards optimal mapping from activation values to firing rates," in *Proc. ICML*, ser. PMLR, vol. 202, 2023.

[5] T. Bu *et al.*, "Inference-scale complexity of spiking neural networks," in *Proc. IEEE/CVF CVPR*, 2025.

[6] D.-M. Ngo, D. Lightbody, A. Temko, C. C. Murphy, and E. Popovici, "HH-NIDS: Hardware heterogeneous network intrusion detection system using MAX78000," *Future Internet*, vol. 15, no. 1, p. 9, 2022.

[7] W. Zahm, G. Nishibuchi, A. Jose, S. Chelian, and S. Vasan, "Low-power cybersecurity attack detection using deep learning on neuromorphic technologies," CSIAC, Tech. Rep., 2024.

[8] Z. Wang *et al.*, "An efficient intrusion detection model based on convolutional spiking neural network," *Scientific Reports*, vol. 14, p. 7054, 2024.

[9] S. N. Prajwalasimha *et al.*, "Event-driven intrusion detection systems using spiking neural networks for edge and IoT security," in *Proc. IEEE ICSCSA*, 2025.

[10] M. G. Karthik *et al.*, "Energy-efficient intrusion detection with a protocol-aware transformer–spiking hybrid model," *Scientific Reports*, vol. 16, p. 7095, 2026.

[11] STMicroelectronics, *ST Neural-ART NPU Concepts*, 2025, uM3225.

[12] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE CISDA*, 2009, pp. 1–6.

[13] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in *Proc. MilCIS*, 2015, pp. 1–6.

[14] STMicroelectronics, *How to Optimize Low-Power Modes on STM32N6 MCUs*, 2025, application Note AN5946.